1.

类 `Stack` 维护两个经典栈类型的元素 `val` 和 `pre`.

```
Stack :: push(x) :
    val.push(x)
    if not pre.empty() then
        x ← min(x, pre.top())
    pre.push(x)

Stack :: pop() → int :
    x ← val.top()
    val.pop()
    pre.pop()
    return x

Stack :: top() → int :
    return val.top()

Stack :: getMin() → int :
    return pre.top()
```

2.

设 a_n 表示 n 辆火车时的答案, $a_0 = 1$. 枚举最后一辆出站火车的编号:

$$a_n = \sum_{k=1}^{n} a_{k-1} a_{n-k}.$$

设其 GF $F(z) = a_0 + a_1 z + \cdots$, 显然

$$F = zF^2 + 1 \Rightarrow F = \frac{1 - \sqrt{1-4z}}{2z},$$

其中正根号解在 0 邻域无上界, 舍去. 利用 Newton 二项式定理提取系数得

$$\begin{aligned} a_n &= -\frac{1}{2}[z^{n+1}](1-4z)^{\frac{1}{2}} \\ &= -\frac{1}{2}[z^{n+1}]\sum_{k=0}^{+\infty}\binom{\frac{1}{2}}{k}(-4)^k z^k \\ &= -\frac{1}{2}\binom{\frac{1}{2}}{n+1}(-4)^{n+1} \\ &= \frac{(2n)!}{n!(n+1)!}. \end{aligned}$$

3.

(1) 撤销:

```
if (!S1.empty()) {
    x = S1.pop();
    restore(x); // restore x's effect on document
    S2.push(x);
}
```

恢复:

```
if (!S2.empty()) {
    x = S2.pop();
    perform(x); // perform x on document
    S1.push(x);
}
```

(2) 从栈底到栈顶, `S1 == { A, B, D }`, `S2 == {}`.

4.

(1) 用一个队列充当容器空间保存栈内元素, 另一个序列保持为空. 在需要弹出时, 将除栈顶外的其他元素移入另一个序列.

$$
\begin{array}{l|l}
1 & \textbf{function}\ \ \mathrm{push}(x):\\
2 & \quad\ \mathrm{enQueue}(Q1, x)\\
 & \\
3 & \textbf{function}\ \ \mathrm{pop}():\\
4 & \quad\ \textbf{repeat}\\
5 & \qquad\ x \leftarrow \mathrm{deQueue}(Q1)\\
6 & \qquad\ \textbf{if}\ \mathrm{isEmpty}(Q1)\ \textbf{then}\\
7 & \qquad\qquad \mathrm{swap}(Q1, Q2)\quad \texttt{// pointer swap}\\
8 & \qquad\qquad \textbf{return}\ x\\
9 & \qquad\ \mathrm{enQueue}(Q2, x)
\end{array}
$$

(2) `push` 复杂度 $\mathcal{O}(1)$, `pop` 复杂度 $\mathcal{O}(n)$, n 是当前栈的大小.

(3) 依次返回 $3, 4, 2$.